A3: Spotify Browser in Angular

11/11/2026

10 Points Possible

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

10/28/2026

Details

Overview

This assignment may be completed in pairs (exactly two students enrolled in the class) or individually. You may choose your own partner or use Slack to help find one. Because the assignment features build on top of one another, it is recommended that you pair program rather than distribute the assignment features. There is no grade benefit or penalty to completing the assignment individually.

In this assignment, you'll demonstrate your ability to (1) gather data from an API, and (2) develop a frontend interface which displays the data. You'll use Angular to create a webpage which communicates with an backend server written in Node.js/Express to browse the music on Spotify, a popular music streaming app. The provided backend server will handle authentication via OAuth and communicate with the Spotify API, but you will have to make HTTP requests to the backend server to trigger API calls. Your page will support searching for artists, albums, and tracks and navigating between the three resources.

By the end the of day on Monday, Nov 2nd, please navigate to the "People" tab on Canvas to sign up for an A3 group (the set named "A3 Pairs", ***NOT*** "Grading Group"). Do correspond with your partner while signing up for groups. If you're the first member to join a group, please sign up for the next group that's empty. Keep a note of the Assignment Group Number that you've signed up for and add it to your readme file.

You will not be able to swap to another group after this date. If something happens after you sign up and you decide you're best off working alone (team conflicts, incompatible schedules, etc.), you can disband your group and let the course staff know. However, you will not be able to team up with someone else, even if they do not have a partner. You will need to complete the assignment alone.

Starter code

Fork the starter repository from GitHub here (https://github.com/travislindenreid/a3-spotify) . Note that if you complete the assignment as a pair, the privacy settings of the repository are such that you will not be able to add them yourself. Instead, one person can create the repository, and the other can join the pre-existing team by finding the team name on GitHub Classroom.

Repository structure

The repository contains somewhere around 80 files. You will **not need to edit most of them** in this assignment.

The repository contains two folders: `webserver` and `client`.

- `webserver` includes the Express/Node.js backend for communicating with the Spotify API.
- `client` includes the Angular frontend for browsing music.

In the frontend (`client` folder), the files which need editing are all under the `app` folder.

Within that folder, **the files/subfolders you should edit are:**

- The six **components** in the `components` **folder**:

    - `about`, `carousel`, `carousel-card`, `search`, `track-list`, and `thermometer`.
    - Each component folder contains four files (`*.component.css`, `*.component.html`, `*.component.ts`, and `*.component.spec.ts`).
        - Of those, all edits will be made in the `.component.html` and `.component.ts` files.
- Three of the four **components** in the `pages` **folder**:
    - `album-page`, `artist-page`, and `track-page`. `home-page` will not be edited.
    - Again, all edits will be made in the `.component.html` and `.component.ts` files.
- The one service in the `services` folder: `spotify-service`. All edits will be made in the `.service.ts` file, none in the `.service.spec.ts` file.

The `webserver` folder is only needed if you aim to complete the extra credit. However, **you will need to download the `dummy_webserver` folder from Canvas, available here (https://canvas.eee.uci.edu/courses/85916/files?preview=37506639)** . We do not include this folder in the repository because it's large and contains data from Spotify that they may not want shared publicly.

You will also edit the `readme.txt` file in the root folder.

This assignment can be completed without writing any additional files or functions. But you may find it helpful to add a file or function, such as to complete one of the bonus features.

Setting up your Workspace

If you're feeling unfamiliar with the command line, you might find the guides in the **resources (https://inf133-wi26.depstein.net/resources)** tab helpful.

Some of the packages/libraries depend on newer versions of Node JS. Check what version of Node JS you have installed with `node --version`. If you installed Node JS before this class, make sure you've updated to at least 10, such as with **nvm (https://github.com/nvm-sh/nvm?tab=readme-ov-file#installing-and-updating)** . If you installed Node during A2, you should not need to update, but it's probably worth checking the version number anyways.

There are no global dependencies required to run the Express webserver.

For the client, **you will need to install Angular through npm**. To do this, you will install the Angular **Command Line Interface (CLI) (https://angular.dev/tools/cli)** globally with:

```
npm install -g @angular/cli.
```

The Angular project is already set up in the `client` folder, but the CLI is necessary to run the project.

You will also need to **install the dependencies for both the webserver and the client**. These dependencies are defined in each project's respective `package.json` files. `cd` into each respective project's folder and install the dependencies with:

```
npm install
```

You may encounter a few installation warnings or high severity security vulnerabilities. We will **ignore them for this assignment**, but these should typically be addressed when using libraries.

When installing, if you run into issues which look like "permission denied", try changing **the permissions of the node_modules directory (https://github.com/angular/angular-cli/issues/6848#issuecomment-386120663)** . If you are still having trouble, ask on Slack or talk to the course staff. It's important to try to get the setup working sooner rather than later, even if you do not plan on working on the assignment until close to the deadline.

You may get a warning that the version of Angular that you have installed is newer than the version of the project. You can safely ignore that in this assignment. Similarly, you may get warnings about deprecated functions or libraries, which you can also ignore.

Running the webserver

We used to be able to access Spotify's servers directly in this course, but after a data breach in 2025, you now have to have a paid Developer account. To avoid requiring you to pay money to do assignments in this course, we will run a dummy webserver instead.

Download the `dummy_webserver` folder from Canvas as described above, unzip it, and continue below using that folder. The dummy webserver does not have all of Spotify's data, but instead includes 10 popular artists: Carly Rae Jepsen, Taylor Swift, Drake (ew, sorry), Billie Eilish, The Weeknd, Ariana Grande, Dua Lipa, Ed Sheeran, Kendrick Lamar (the GOAT), and Adele. If an unknown ID is requested, the server returns placeholder data to prevent crashes.

`To get started, cd` into the webserver folder and **install the dependencies** with npm install. Then, to **run the Express webserver**, run:

```
npm start
```

This will start the webserver at `127.0.0.1:8888`. If you navigate to this root page (`http://127.0.0.1:8888/`) in your browser, you'll likely get a error in your console suggesting that "No default engine was specified". This is because no endpoint is defined for the root page (see the `index.js` file to see the defined endpoints). If you open endpoints which are defined, like `http://127.0.0.1:8888/login`, you shouldn't get this error.

If you make a change to code in the webserver (which you should *not need to do* to complete the required portions of this assignment), you will have to end the running program and re-run it. End it by typing `Ctrl-C` into the command line and then re-running it with `npm start`.

Running the client

To run the Angular client, `cd` into the client folder and run:

```
ng serve
```

This will start the client at `localhost:4200`. Adding the `--open` flag will open it up in the browser. Be sure the dependencies have been installed first via `npm install`.

Any code changes will be automatically reloaded on the client.

This is what your localhost:4200 page will look like before you begin coding:

Working in pairs

If you are completing the assignment in pairs, you should **work in one shared GitHub repository**. The easiest way to do this is to have one of you create the repository and **add the other as a collaborator (https://help.github.com/articles/inviting-collaborators-to-a-personal-repository/)** .

The downside of working in pairs is that you open yourself up to potential issues merging one person's code into another. Make sure you always **update the version of the code on your computer (https://help.github.com/desktop/guides/contributing-to-projects/syncing-your-branch/)** prior to beginning work. This **guide (https://gist.github.com/ccannon94/a75f1f725d33a1834dd7f5feebbc7d4b)** offers some suggestions for how to resolve potential conflicts which might arise. But more generally, we suggest that you complete this assignment via pair programming, alternating roles as drivers and navigators.

Using another framework

A few students have asked the course staff whether they can use another framework, such as React, to complete this assignment in order to pick up a new skill. **While this is an option, you are required to get approval from the course staff prior to pursuing this option.** We expect to only allow grant approval for students who have used Angular, React, or another framework already. The next assignment will implement a mobile app in Ionic, a framework which adds mobile components on top of the Angular framework. Weak Angular fundamentals could therefore cause problems later in the course.

Should you choose to use another framework, it is done with the understanding that the course staff will not support you in learning the framework or debugging your code. The course staff will not be more lenient in grading or give any extra credit for using something not covered in the course. You are required to complete the same assignment, which will likely mean adapting the provided starter code to the new framework. Should you choose this path, **you are on your own**.

Please include in your `readme.txt` any information we would need in order to run your code.

Requirements

There are five parts to this assignment. The later parts use many of the same components, so once those are implemented, the later parts become a matter of gluing them together. You can also optionally add one of a number of bonus features.

We recorded a **demo (https://www.awesomescreenshot.com/video/49220355?key=6641ac78c0e515385bab88dd6c4ce51d)** of what your Spotify browser will look like once you have finished. The demo shows how we expect the browser to respond to pressing buttons or making searches. The solution may be slightly different visually from yours. Spotify regularly changes its search results, so it's also expected that you may receive slightly different results than in the demo.

Part 1: Communication with the (dummy) Webserver (1 point)

When using the dummy webserver, there is no need to log in. The "login" button will simply redirect you back to the client, and does not need to be implemented.

However, in order to make API requests, you will need to **add some functionality to the Spotify Service** (services/spotify.service.ts). First, update the sendRequestToExpress function in the Spotify service to ask the Express webserver to make an API request to Spotify at the endpoint passed in. The endpoints should then return the appropriate data type for each API call. You will be able to test your function by building out the pages in the assignment. Given the server changes, the 1 point is simply for demonstrating any communication with the dummy webserver on your pages.

Depending on how you wrote your code for A2, this might be the first time you've tried to access member variables inside of callback functions. The this keyword gets overwritten if you use the traditional function syntax (function()), while it gets preserved (keeps the reference to the parent class) for arrow functions (() =>). For the assignment, you'll therefore probably want to use arrow functions.

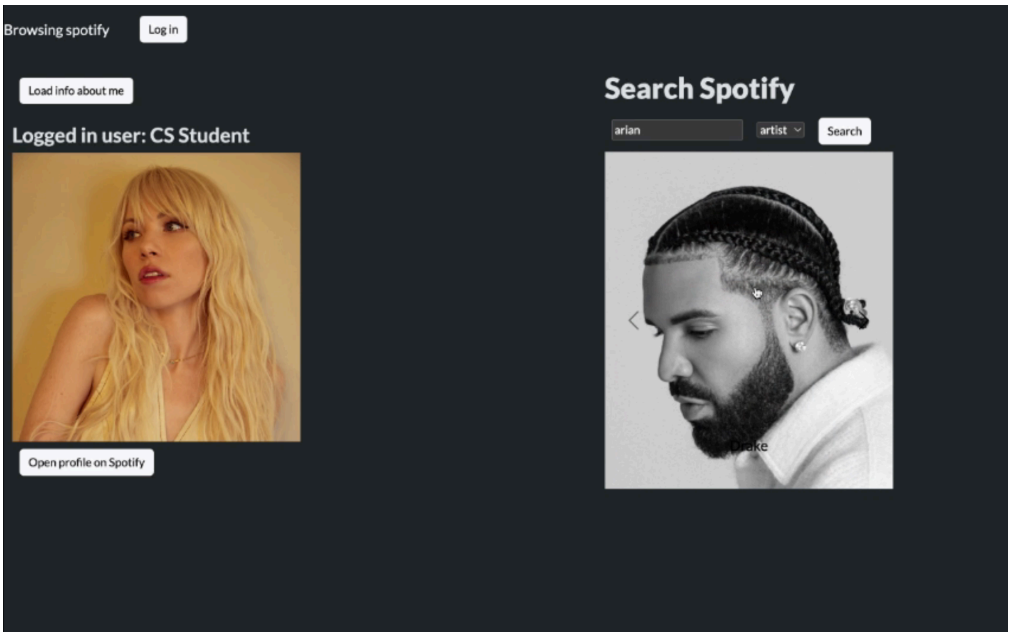
Part 2: Spotify Browser Home Page (4 points)

Screenshots from here down were made via Spotify's API. For an example of what the homepage, search results, etc. should (presumably) look like with the dummy webserver, please refer to **this video (https://www.awesomescreenshot.com/video/49220355?key=6641ac78c0e515385bab88dd6c4ce51d)** .

In this part, you will create a homepage to:

- view **information about the dummy "logged in user" account**, and
- create **search requests to the Spotify API.**

As you get started with Angular, it may be helpful to review the code from the demos or the lecture recordings.



Populating Information About the User (1 point)

Edit the `about` component to call the endpoint created for the "about me" page when the load info button is clicked. To do so, you will need to inject the Spotify service you just edited. The **Spotify API endpoint reference (https://developer.spotify.com/documentation/web-api/reference/)** describes what this endpoint and other endpoints return. Update the variables in the component's `.ts` file and bind them to the appropriate places in the `.html` file. This will populate the left side of the home page with the logged in user's name, profile picture, and a link to open their profile on Spotify.com.

Populating the Search Component (3 points)

The right side of the page contains the `search` component. The search component contains an input and a select (dropdown) menu to declare what to search for. Bind the values in the component's model to the input and select controls. Bind the search button to a function which uses the Spotify service to have Express make the API request. You can also search online to learn how to trigger a search when a user presses "enter" in the input field or changes the dropdown menu. These two features are optional--you are only required to bind the search button. For simplicity, your code can assume that any search typed in is valid (e.g., returns at least one artist/track/album). Your functionality may also break if you search, but then switch what category is being searched for (artist, track, album). Correcting this is also optional.

Map the response from Express into an array of the appropriate resource type--ArtistData, AlbumData, or TrackData. To do so, you will have to update the Spotify Service to create objects based on the category being searched for. Similar to the example given for parsing ProfileData, the constructor of each resource can automatically parse the dictionary values Spotify returns into an Object. For example, AlbumData loads the genres of the Album and creates an array with each of the artists on the album. Note that all three of these resource classes extend the ResourceData class, which parses some dictionary values common across all the resource types. You should not need to edit any of these four files; contact the course staff if a response parses differently than you would expect or causes an error.

Once the array of resources has been created, display the search results in either a `carousel` component (for artists or albums) or a `track-list` component (for tracks).

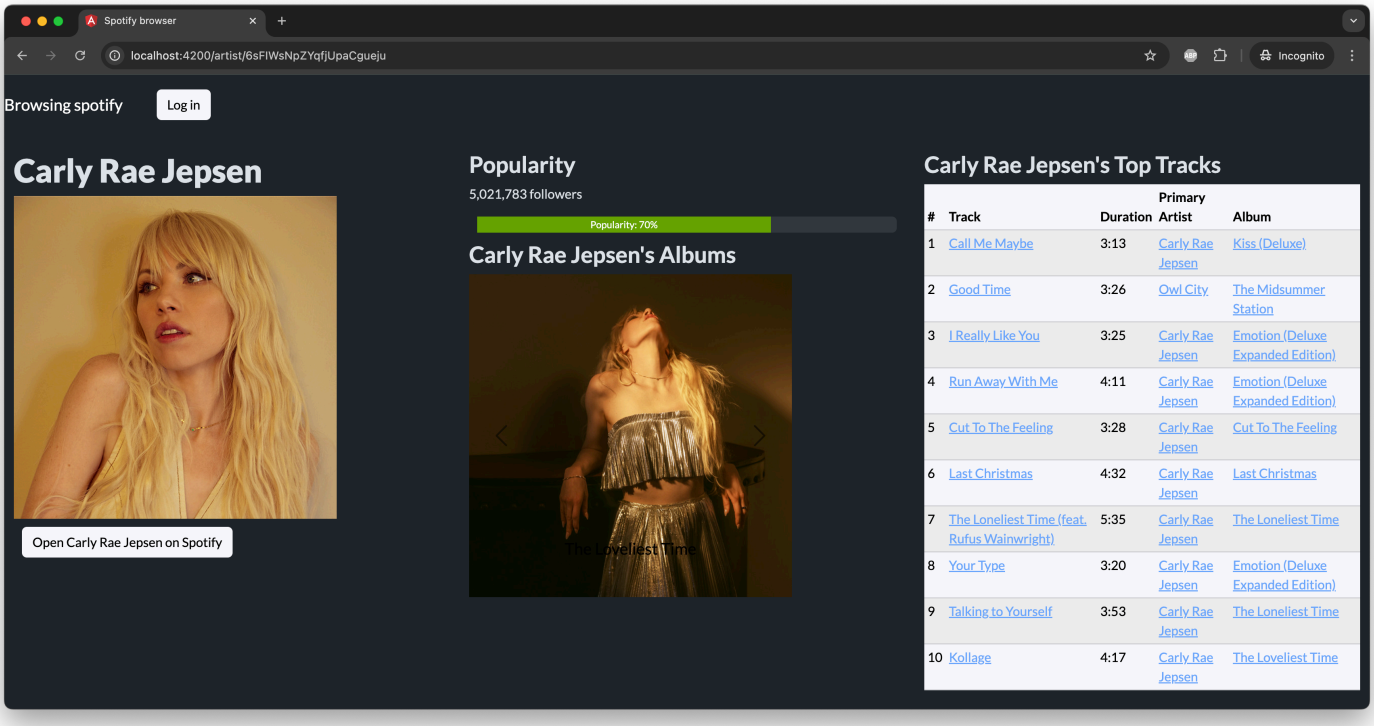
The carousel is implemented via a **Bootstrap carousel (https://getbootstrap.com/docs/5.3/components/carousel/)** . Edit the carousel component to create `carousel-card` components for each resource passed in. Additionally, edit the carousel-card component to link to the local URL for browsing the specific artist or album. Check the app router (`app-routing.module.ts`) to identify what the URL should be.

The track list is implemented in the `track-list` component. Edit the component to display each track in the response as a row in the component's table, displaying the track's index, name, duration, artist, and album. The component should support hiding the artist and/or album columns for the track when the hideArtist or hideAlbum variables are true.

Beginning in this part, some of the links in your Angular app will direct to other URLs or endpoints within your Angular app, while a few will link to pages on Spotify.com. There is one link to Spotify.com associated with each page (*Open profile on Spotify*, *Open [artist name] on Spotify*, etc.), which includes the text *on Spotify*. All other links should route a person to other pages within the app. For example, all artist links in the `carousel-card` component should send a person to the Artist Page (or Album pages if the carousel is showing albums). The name of all tracks in the `track-list` should direct a person to the corresponding Track Page, while the Primary Artist and Album links should direct a person to the respective Artist and Album pages.

Part 3: Spotify Browser Artist Page (3 points)

In this part, you will create a page to view some information about a searched-for or linked-to artist. You will update the `artist-page` component to do this. For simplicity, you can assume the id passed via the URL is always valid (e.g., always refers to an artist which exists in the Spotify API). The artist-page includes code for retrieving the artist's id from the URL.



Update the Artist Page to retrieve the basic information about the artist, their top tracks, and their albums. The Express webserver has endpoints for each of these three requests. Update the component to display the information as described in the component's template. You will re-use the `track-list` and `carousel` components on this page.

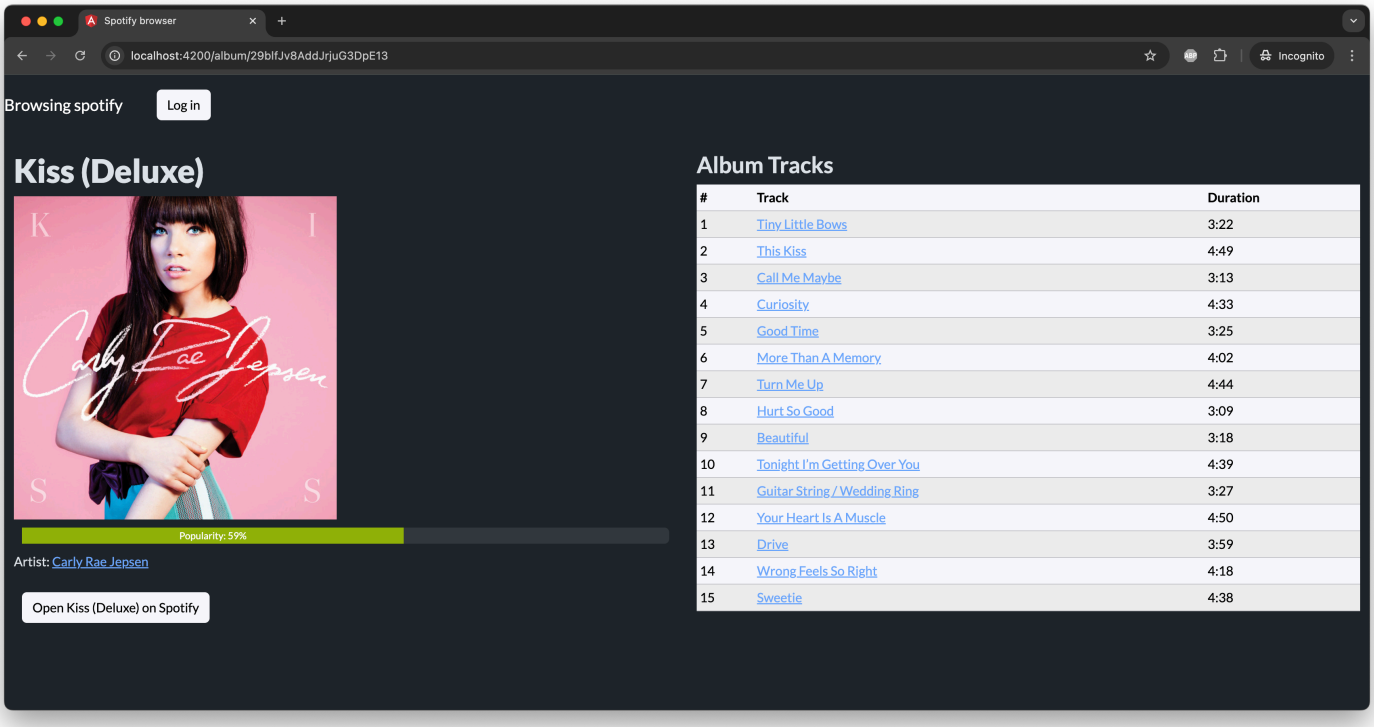
You will also have to edit the `thermometer` component to display the artist's popularity. The thermometer uses a **Bootstrap progress bar (https://getbootstrap.com/docs/5.3/components/progress/)** to display the track's audio features. Note the component does not have any Input fields, so you will have to identify what fields you need and define them.

If you want (optional), you can edit the `popularity` class to use the **chroma library (https://gka.github.io/chroma.js/)** to display the progress bar's color as a mix between two values depending on it's percent. The provided mixes between red and green. You can choose whatever colors you like, so long as differences are visually distinct. HSL blending is recommended, as it more closely models how human vision blends colors.

You will again have to update endpoints in the Spotify Service to get information about an artist, their top tracks, albums, etc.

Part 4: Spotify Browser Album Page (1.5 points)

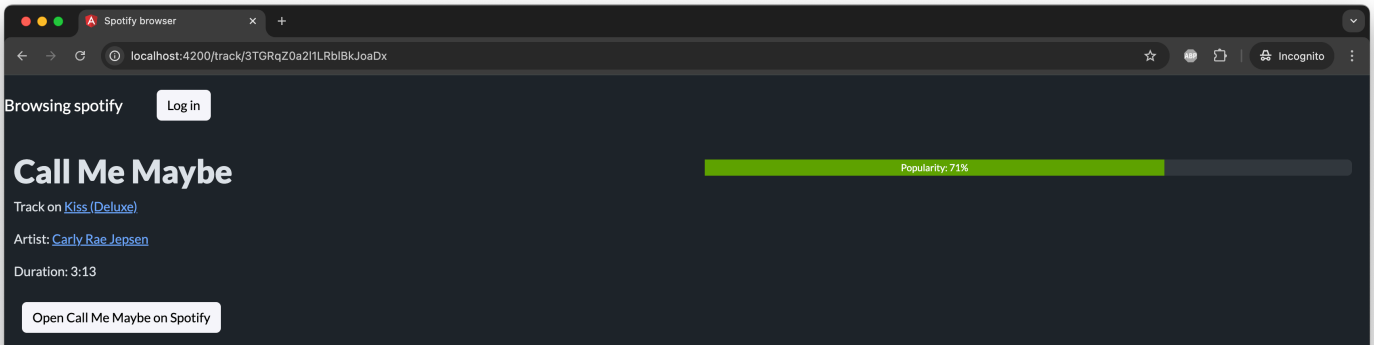
In this part, you will create a page to view some information about a searched-for or lined-to album. You will update the `album-page` component to do this. For simplicity, you can assume the id passed via the URL is always valid (e.g., always refers to an album which exists in the Spotify API). The album-page includes code for retrieving the album's id from the URL.



Update the Album Page to retrieve the basic information about the album and its tracks. The Express webserver has endpoints for each of these two requests, which you will again need to call in the Spotify service and parse data from. Update the component to display the information as described in the component's template. You will re-use the `track-list` and `thermometer` components on this page.

Part 5: Spotify Browser Track page (0.5 points)

In this part, you will create a page to view some information about a searched-for or lined-to track. You will update the `track-page` component to do this. For simplicity, you can assume the id passed via the URL is always valid (e.g., always refers to a track which exists in the Spotify API). The track-page includes code for retrieving the track's id from the URL.



Update the Track Page to retrieve the basic information about the track and its popularity. The Express webserver has endpoints for each of these two requests. Update the component to display the information as described in the component's template. You will re-use the `thermometer` component on this page.

Optional/Bonus Features

The five parts of the assignment create a tool for browsing the Spotify API. There are many ways of extending this browser, and we can offer one point of extra credit for implementing an extension. We've enumerated some potential extensions:

- The webserver for this assignment can only handle a single user because tokens are being saved in a JSON file with no user information. Use **session middleware (https://github.com/expressjs/session)** and an improved scheme for storing tokens to

handle multiple users.
- Adding a component which plays music for an Artist/Album/Track using Spotify's **Web Playback SDK. (https://developer.spotify.com/documentation/web-playback-sdk/)** You can add the player to any page within your Spotify browser.

These extra credit options require a Spotify Premium subscription, and require leveraging the "non-dummy" webserver. Please be sure to indicate in your readme whether you completed an extra credit feature, and consider recording a video of your extra credit working (such as `extra_credit.mp4`).

We are open to other suggestions for bonus features.

Submission

Submit a .zip file containing your repository here on Canvas. Only one team member needs to submit.

Any late uploads are subject to the course's **late policy (https://inf133-wi26.depstein.net/syllabus#duedates)** . Your project is probably quite large; please remove the `node_modules` folder from both the client and the webserver prior to zipping. As discussed in lecture, the advantage of using npm is that we can reference your `package.json` and `package_lock.json` to download the same packages and libraries that you used on your computer. **We may apply a penalty for including these files**, as it slows down our process of grading. You can exclude the `webserver` and `dummy_webserver` folders from your submission, but we will not penalize for including them so long as you remove your `node_modules`.

Please update your `readme.txt` with how long the assignment took, who or what online resources you consulted with, any bonus features you added, and anything else we should know. If the assignment was completed in a pair, the readme should list names/emails/ids of both students. Only one submission needs to be made, however. **We may apply a penalty for failing to include your name(s) in the readme**, as it slows down our process of grading.

Because many people will complete this assignment in pairs, **you are also required to complete a short evaluation on Canvas signifying how work was distributed**. The **survey is available on Canvas here (https://canvas.eee.uci.edu/courses/85916/quizzes/465194)** . Please fill out the survey even if you worked alone.

Grading

This assignment will be graded on a scale of 10 points, broken down as follows:

- Communication with the Webserver (1 points)
- Spotify Browser Home Page (4 points)
- Spotify Browser Artist Page (3 points)
- Spotify Browser Album Page (1.5 points)
- Spotify Browser Track Page (0.5 point)

A bonus feature completed will earn you 1 point of extra credit (e.g., no points given for beyond one bonus feature). The maximum grade for this assignment is therefore 11/10.

The course staff reserves the right to redistribute, penalize, or award bonus points to one or both students in situations where work was not distributed equitably among partnerships.

In prior courses, you've been asked to follow good principles for indentation, naming variables, commenting, etc. We expect you to do the same in this course, but aim to avoid being draconian in our enforcement of these principles. Egregiously poor formatting, completely uncommented code, etc. may incur a small penalty (e.g., -1 point), but we expect this to be rarely applied.

View Rubric

A3	
Criteria	**Ratings**

Criteria	Ratings					
Spotify browser home page view longer description	**All features** 4 pts	**one feature missing** 3.5 pts	**one feature missing** 3 pts	**multiple missing features** 2.5 pts	**multiple missing features** 2 pts	**mu mi fea**
Spotify browser artist page view longer description	**all features present** 3 pts	**one feature missing** 2.5 pts	**multiple missing features** 2 pts	**multiple missing features** 1.5 pts	**n**	
Spotify browser album page view longer description	**finished** 1.5 pts	**multiple missing features** 1 pts	**multiple**			
Spotify browser track page view longer description	**Works** 0.5 pts					
Communication with the webserver view longer description	**Works** 1 pts					